



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{ 'wall_ahead': True/False, 'food_here': True/False }` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A lookup table for Chess would need $|P|^T$ entries — with $\sim 10^{40}$ legal states and $\sim 10^{120}$ possible game sequences (Shannon Number), no storage system could hold it and no one could write it. As the agent's lifetime T increases, the table grows exponentially, making it worse the longer the agent runs. This is why we write Agent Programs (algorithms) instead of lookup tables.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

```
if percept.get('food_here'): return 'Suck'  
if percept.get('wall_ahead'): return 'Left'  
return 'Up'
```

Each is a direct condition → action mapping with no `__init__` and no memory — every call starts fresh, exactly matching the lecture's "zero memory" definition of a Simple Reflex Agent.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The simulator output shows the same percept (`wall_ahead=True`, `food_here=False`) and same action ('Left') repeating for all 15 steps, frozen at (0,0). This happens because Partial Observability removed `agent_pos`, so the agent can't tell where it is, two different failed situations look identical. Combined with no Percept History (no memory of what it tried before), the agent's decision is a pure function of the current percept only. Since the percept never changes, the action never changes, an unbreakable infinite loop, matching the lecture's vacuum-robot bounce example.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The lecture defines the Transition Model (how the world evolves) and Sensor Model (how the agent's actions affect the world).

In code:

```
self.percept_history.append(dict(percept))    # Transition Model  
self.percept_action_memory[signature] = action  # Sensor Model
```

percept_history tracks what the world currently shows; percept_action_memory tracks the outcome of the agent's own past actions per situation. This is what let the agent try Left on step 1, recognize the repeat on step 2, and switch to Right to escape to (1,0) — something the memoryless Simple Reflex Agent could never do. One limitation: memory is keyed only on percept signature, not exact position, so the agent can still revisit cells (e.g. (1,3)) — it solves the immediate loop but isn't a full spatial-navigation fix, matching the lecture's noted limitation for Model-Based agents.

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING